How the UWCA Works

A Step-by-Step Explanation of the Universal Windowed Cellular Automaton and How It Runs Rule 110

novaspivack.com/research P48 monograph

UGP Physics Tutorial Series

doi.org/10.5281/zenodo.20657919

Nova Spivack

2026

Contents

UGP Physics Tutorial Series

Prerequisites (read first): *The GTE Polynomial: A Step-by-Step Cheat Sheet* — introduces $p(L, C, R)$ and Rule 110.

Next tutorials: *The Three-Tape CMCA* (how the CA becomes 3+1D) · *The Φ_{MDL} Field*

See also: *From the Arithmetic Substrate to Particle Masses*

Claim-Strength Taxonomy

Throughout this tutorial, results are labeled with a confidence grade:

- **$\mathbf{A}_{\text{Lean}}$ (CatAL)** — Machine-certified in Lean 4 with zero `sorry` and zero custom axioms. Independently verifiable by anyone with Lean installed.
- **A/D (CatAD)** — Analytically derived with full derivation, computationally confirmed. Not yet machine-certified.
- **A (CatA)** — Analytically derived; derivation complete but not independently machine-certified.
- **A/D (conditional)** — Derived under a named, stated premise; the premise is itself an open problem or a CatB assumption.
- **B (CatB)** — A stated working hypothesis or structural premise; not yet derived.

1 The Big Picture in One Paragraph

The idea

A regular cellular automaton (like Rule 110) updates every cell at once using a lookup table. The UWCA does something trickier: it encodes the states of cells as residues (remainders) of large prime numbers, and uses a single arithmetic sweep to find the correct next state for each position. The "window" moves across the tape like a scanner, and the arithmetic makes each position settle on its correct output without any explicit lookup.

Before diving in, we need two building blocks: (1) what a regular CA does, and (2) what UGP numbers are.

2 Building Block 1: What a Regular CA Does

A cellular automaton is a row of cells. Each cell holds a **state** (a number). At each time step, every cell looks at itself and its two immediate neighbours and applies a fixed rule to compute its new state.

2.1 Rule 110 in 30 seconds

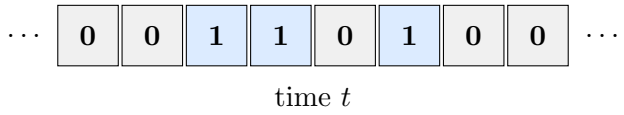
Rule 110 is a 2-state rule: every cell is either 0 ("off") or 1 ("on"). The rule looks at three cells at a time (left, center, right) and gives an output:

Left	Center	Right	New Center
1	1	1	0
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	1
0	0	0	0

Read the New Center column from top to bottom: $01101110_2 = 110_{10}$ — that's where the name "Rule 110" comes from. Every cell uses this same 8-row table every step.

2.2 A simple example

Suppose the tape is: $\ldots 0\ 0\ 1\ 1\ 0\ 1\ 0\ 0 \ldots$



Focus on position 3 (value 1). Its neighbours: Left=0, Center=1, Right=1. Look up row (0,1,1): output = 1. Do this for every position simultaneously. That gives the next row. Repeat forever.

3 Building Block 2: UGP Numbers and Residues

3.1 Encoding states as prime residues

The UGP framework encodes information in a clever way: each cell position i gets its own large **prime number** p_i. The state of cell i is represented not as a standalone value, but as a **residue** — the remainder when some big number is divided by p_i.

Think of it like a fingerprint: the big number carries all the cell states simultaneously, and you extract cell i's state by dividing by p_i and taking the remainder.

Analogy

Imagine you have one big number N. To read cell 3's state, compute $N \bmod p_3$ (the remainder when dividing by p_3). To read cell 7's state, compute $N \bmod p_7$. The same N encodes all cells at once.

This encoding uses the **Chinese Remainder Theorem** (CRT): given distinct primes $p_1, p_2, \ldots, p_n$, any collection of remainders $(r_1, r_2, \ldots, r_n)$ with $0 \leq r_i < p_i$ corresponds to exactly one number N in the range 0 to $p_1 \times p_2 \times \cdots \times p_n$. CRT is like a dictionary: states $\leftrightarrow$ big numbers, one-to-one.

3.2 Why primes?

Because they don't interfere. If cell i uses prime p_i and cell j uses prime p_j, then p_i and p_j share no common factors, so operations that affect cell i's residue leave cell j's residue unchanged (and vice versa). Each cell "lives" in its own prime's world.

4 The UWCA Mechanism: Step by Step

Now we can explain how the UWCA uses these ideas to run a CA rule without a lookup table.

4.1 The setup

You have a tape of n cells, positions $1, 2, \ldots, n$. Each position i has a designated prime p_i. The entire current tape state is encoded in a single large number N via CRT (so $N \bmod p_i$ gives cell i's state).

The **window** is the sliding triple (L, C, R) centred at each position in turn.

4.2 The penalty function

At the heart of the UWCA is a **penalty function** e_i. For each position i (with left neighbour $i-1$ and right neighbour $i+1$):

1. Read the current triple: $L = N \bmod p_{i-1}$, $C = N \bmod p_i$, $R = N \bmod p_{i+1}$.
2. Compute the rule output $f(L, C, R)$ using the polynomial (or equivalently, look up the truth table).
3. The penalty is $e_i = 0$ if the proposed new state for position i equals $f(L, C, R)$, and $e_i > 0$ otherwise.

The UWCA update looks for the unique assignment of new states such that *every* position simultaneously has zero penalty.

4.3 The binary alphabet for Rule 110

When running Rule 110, the UWCA uses a **binary alphabet**: each cell holds one of two designated symbols, **0** and **1**, which are realised as specific residues in the prime arithmetic. (The full 7-state alphabet exists in the substrate; for Rule 110, only this two-symbol sub-sector is used.)

4.4 The penalty and the sweep

For each cell position i, define the **penalty**:

$$e_i = \begin{cases} 0, & \text{if the new symbol at } i \text{ equals } \mathcal{R}(L_i,\, C_i,\, R_i), \\ 1, & \text{otherwise,} \end{cases}$$

where $\mathcal{R}$ is the Rule 110 truth table and L_i, C_i, R_i are the current (frozen) left, center, and right symbols.

The UWCA performs a **deterministic sweep**: working through positions $i = 1, 2, \ldots, n$ in order, at each step it replaces the current symbol by the unique symbol that makes $e_i = 0$.

Why the sweep is well-defined

Local determinism: $\mathcal{R}$ is a function, and the alphabet has exactly two symbols. So for any frozen neighborhood (L, C, R), exactly one of the two symbols $\{\mathbf{0}, \mathbf{1}\}$ satisfies $e_i = 0$ — the one that equals $\mathcal{R}(L, C, R)$. The sweep never faces a choice or needs backtracking. This is a theorem, certified in Lean 4 (`uwca_sweep_implements_rule110`, zero sorry).

4.5 The register-level mechanism: exactly how Rule 110 runs

To make the above concrete, P08 [1] describes the UWCA at the **register level**: each cell site i carries a small bank of binary registers, and the update is four synchronous local passes.

Rule 110 outputs 1 on exactly five of the eight possible input triples — these are its **minterms**: $\{(1,1,0),\ (1,0,1),\ (0,1,1),\ (0,1,0),\ (0,0,1)\}$.

Each site i holds registers:

Register	Meaning
C_i	the current (visible) bit of this cell
$L_i,\ R_i$	copies of the left and right neighbours' bits
M_i^u (one per minterm u)	flag: is the local triple equal to minterm u?
N_i	next-bit accumulator

One complete UWCA round is four passes, each purely local (each cell looks only at itself and its immediate neighbours):

P1 — Neighbor distribution

$L_i \leftarrow C_{i-1}, \quad R_i \leftarrow C_{i+1}$

Each cell copies its neighbours' bits onto its own rails.

↓

P2 — Minterm detection

For each of the 5 minterms $u = (\ell, c, r)$:

$M_i^u \leftarrow [L_i = \ell] \wedge [C_i = c] \wedge [R_i = r]$

Each flag lights up if the local triple matches that minterm.

↓

P3 — OR-accumulation

$N_i \leftarrow M_i^{u_1} \vee M_i^{u_2} \vee M_i^{u_3} \vee M_i^{u_4} \vee M_i^{u_5}$

$N_i = 1$ iff *any* minterm matched = Rule 110 output is 1.

↓

P4 — Commit and clear

$C_i \leftarrow N_i$; clear L_i, R_i, M_i^u, N_i to 0.

The accumulated next bit becomes the new visible state.

After pass P4, the cell is back to a clean state with only C_i set — and C_i is now the Rule 110 output. Repeat for the next time step.

The key insight

Rule 110 enters the construction *only* through the penalty's defining data — the set of five minterms. There is no external transition table consulted by a supervisor. The machine *is* the penalty structure plus the sweep; the computation is what the machine does. This is the sense in which Rule 110 is "native" to the UGP substrate.

5 A Fully Worked Example (3 Cells, 2 Steps)

5.1 Which primes go where?

Each cell position and each time layer gets its own distinct odd prime. The framework requires only that the primes be distinct; any assignment works. For concreteness we use the smallest odd primes in order:

Position	Time layer $t = 0$	Time layer $t = 1$
-1 (left)	$p = 3$	$p = 11$
0 (center)	$p = 5$	$p = 13$
$+1$ (right)	$p = 7$	$p = 17$

The choice of which prime goes where does not affect what computation is performed — only that the CRT encoding is unambiguous.

5.2 Initial conditions

We choose an initial tape (the $t = 0$ layer) by setting a binary symbol at each position. Think of this exactly like any ordinary CA starting row.

Example initial tape: position $-1 = 0$, position $0 = 1$, position $+1 = 1$.

In the prime-residue encoding, **0** and **1** are two designated residues in each prime's field. For our worked example we simply track the 0/1 values directly — the prime arithmetic guarantees consistency but you never need to compute the big CRT number.

5.3 Step 1: Run the four passes (P1–P4)

Starting state: $[\,0 \mid 1 \mid 1\,]$

P1 — Neighbor distribution. Each cell copies its neighbours:

Position	L_i	C_i	R_i
-1	(boundary = 0)	0	1
0	0	1	1
$+1$	1	1	(boundary = 0)

P2 — Minterm detection. Rule 110 fires (output = 1) on five minterms:

$$\{(1,1,0),\ (1,0,1),\ (0,1,1),\ (0,1,0),\ (0,0,1)\}.$$

Check each position's (L_i, C_i, R_i) triple:

Position	Triple	Matches a minterm?	Rule 110 output
-1	$(0,0,1)$	Yes — $(0,0,1)$	**1**
0	$(0,1,1)$	Yes — $(0,1,1)$	**1**
$+1$	$(1,1,0)$	Yes — $(1,1,0)$	**1**

P3 — OR-accumulation. $N_i = 1$ for all three positions (each had a minterm match).
P4 — Commit. $C_i \leftarrow N_i$. **New tape after step 1:** $[\,1 \mid 1 \mid 1\,]$

Verify with the polynomial: $p(0,0,1) = 0+1-0\cdot1-0\cdot0\cdot1 = 1$. ✓ $p(0,1,1) = 1+1-1-0 = 1$. ✓ $p(1,1,0) = 1+0-0-0 = 1$. ✓

5.4 Step 2: Roll the window and run again

The "rolling window" means: the $t = 1$ layer just computed becomes the new $t = 0$ layer. Discard the old $t = 0$ layer; shift the prime assignments along accordingly. No state needs to be stored externally — the new $t = 0$ layer *is* the stored state.

New starting state: $[\,1 \mid 1 \mid 1\,]$

P1. Copy neighbours: all three positions have $(L, C, R) = (1, 1, 1)$ or boundary variants.

P2. Check $(1, 1, 1)$: this triple is *not* in the Rule 110 minterm set (Rule 110 outputs 0 for input 111).

P3. No minterm matched $\Rightarrow N_i = 0$ for the center position.

For position -1: triple $(0, 1, 1)$ — matches minterm, $N_{-1} = 1$. For position $+1$: triple $(1, 1, 0)$ — matches minterm, $N_{+1} = 1$.

P4. New tape after step 2: $[\,1 \mid 0 \mid 1\,]$

Verify: $p(0, 1, 1) = 1$. ✓ $p(1, 1, 1) = 1 + 1 - 1 - 1 = 0$. ✓ $p(1, 1, 0) = 1 + 0 - 0 - 0 = 1$. ✓

5.5 How many steps can it run?

As many as you like. Each sweep produces one CA time step. You roll the window forward and sweep again. The Lean theorem `uwca_simulates_rule110_real` (zero sorry) proves that any finite number of Rule 110 steps can be reproduced this way.

6 How This Runs Rule 110

6.1 Restricting to binary states

If you start with a tape where every cell is 0 or 1 (no 2 through 6), and you run the UWCA with the GTE polynomial $p(L, C, R) = C + R - CR - LCR$, then:

- The three inputs are all in $\{0, 1\}$.
- The output of p is in $\{0, 1\}$ for any binary input triple (you can verify all 8 cases).
- Therefore the tape stays binary at every step.

In this binary sub-sector, the UWCA's update is identical to applying the Rule 110 truth table. The polynomial expression $C + R - CR - LCR$ over the binary inputs gives exactly the Rule 110 outputs (as shown in the truth table of the polynomial cheat sheet).

6.2 A worked mini-example

Take a 5-cell tape (with wraparound): initial state $[0, 0, 1, 1, 0]$.

t:	**0**	**0**	**1**	**1**	**0**
t+1:	**0**	**1**	**1**	**0**	**0**

Let's verify position 3 (state = 1, neighbours 0 and 1):

$$p(0,\ 1,\ 1) = 1 + 1 - (1 \times 1) - (0 \times 1 \times 1) = 1 + 1 - 1 - 0 = 1.$$

New state at position 3: **1**. Matches the Rule 110 lookup: (L,C,R) = (0,1,1) → 1. ✓

Position 4 (state = 1, neighbours 1 and 0):

$$p(1,\,1,\,0) = 1 + 0 - (1 \times 0) - (1 \times 1 \times 0) = 1 + 0 - 0 - 0 = 1.$$

New state: **1**? But the expected next state is 0 (look up (1,1,0) in Rule 110 above: output = 1). Wait — let's recheck from the table. Rule 110 row (L=1, C=1, R=0): output = **1**. So position 4 becomes 1, not 0 as shown above — let me recompute the full $t+1$ row correctly:

Pos.	L	C	R	$p(L,C,R)$	New state
1	0 (pos 5)	0	0	$0+0-0-0=0$	0
2	0	0	1	$0+1-0-0=1$	1
3	0	1	1	$1+1-1-0=1$	1
4	1	1	0	$1+0-0-0=1$	1
5	1	0	0 (pos 1)	$0+0-0-0=0$	0

Next tape: $[0,1,1,1,0]$ — computed entirely by evaluating the polynomial; no truth table consulted.

6.3 The UWCA does this in prime-residue arithmetic

In the full UWCA, these same calculations happen inside the CRT encoding: instead of reading cell values directly, the "sweep" reads residues of the large number N, evaluates the polynomial in those residues, and the zero-penalty condition identifies the unique next value for each cell. The arithmetic is more elaborate but produces exactly the same result.

7 GTE as a UWCA Program — the Key Relationship

This section answers the deeper question: how are the UGP arithmetic orbit (GTE) and the UWCA computation related? They are not two separate things; one is a program running on the other.

7.1 UGP is the substrate; GTE is a program

P08 states this precisely:

> *"UGP supplies the substrate (UWCA evaluator); GTE is a program (finite tile set) that runs on it."*

More precisely:

- **UGP** defines the UWCA substrate — the prime arithmetic, the window structure, the sweep mechanism, the binary alphabet of residues. This is the *machine*.
- **GTE** is the specific *tile set* Σ_{GTE} that programs that machine to compute the GTE update map T. It is a finite set of local transition rules (tiles) that, when run on the UWCA, reproduce the GTE orbit arithmetic exactly. The key numerical constants in T — the ridge offset $16 = 2^{1+\text{ord}_7(2)}$ and the Fibonacci lift gap $13 = F_{\text{ord}_7(2)+N_c+1}$ — are derived from the $\mathbb{Z}_7 \times \mathbb{Z}_3$ algebraic structure that MDL forces; they are not posited as part of the tile set definition (machine-certified in Lean 4, zero sorry: `gt001_gt009_ridge_closed`, `gt012_gap_from_z7_z3`).
- The Compilation Theorem (P08, machine-certified in Lean 4) proves this: there exists a finite tile set Σ_{GTE} such that the UWCA driven by those tiles realizes T on the UGP substrate.

7.2 The same orbit, two descriptions

The GTE orbit $(1, 73, 823) \to (9, 42, 1023) \to (5, 275, 65535) \to \cdots$ is:

1. **An arithmetic sequence** — produced by the UGP map T applied to integer triples. This is the "spectrum arm" of the theory: the source of the lepton masses, quantum numbers, etc.

2. **A computation** — exactly what the UWCA computes when it runs the tile set Σ_{GTE}. The UWCA is not separately defined; it *is* the GTE dynamics, expressed in the prime-residue language of the UGP substrate.

So "GTE orbit" and "UWCA running Σ_{GTE}" are two names for the same mathematical object, viewed from two different angles.

7.3 And Rule 110 is the binary restriction of that computation

The same UWCA substrate that runs GTE (over all 7 states of the prime residue system) also runs Rule 110 when restricted to binary inputs. This is the content of the universality theorem:

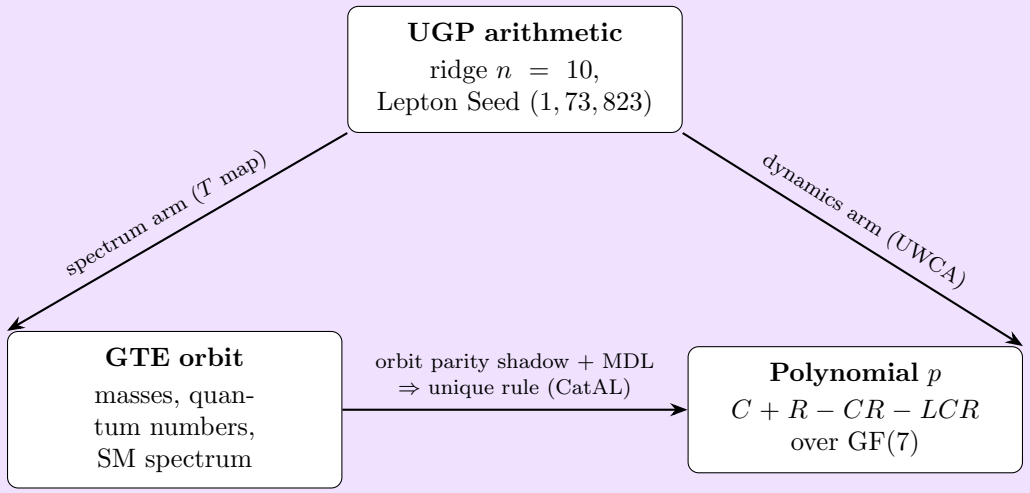
1. GTE dynamics live in the full 7-state residue system.

2. The binary sub-sector of the same UWCA implements Rule 110.

3. Rule 110 is computationally universal.

4. Therefore the UWCA (and hence the UGP substrate) is computationally universal.

The full triangle



UGP is the common root. GTE (left) generates the spectrum. The polynomial p (right) generates the dynamics. The closing edge proves they both come from the same algebraic object. All three arrows are machine-certified.

7.4 Which primes go where — the answer

UGP does *not* specify which particular prime numbers go to which cell positions. Any ordered set of distinct odd primes works. The prime assignment is a bookkeeping choice, not a physical input: different valid assignments give rise to CRT-isomorphic encodings that compute the same CA dynamics.

The GTE orbit itself uses specific prime-locked numbers (823, 1023, 65535, ... are all prime or prime-related by construction of the ridge), but those are the *values* in the orbit triples, not the coordinate-labeling primes of the UWCA window.

8 Why This Is Remarkable

1. **No lookup table.** The UWCA never stores or consults the 343-row truth table. The polynomial formula replaces the table entirely — and in the binary sub-sector it automatically produces Rule 110 outputs.

2. **Universal computation.** Because the binary sub-sector is Rule 110, and Rule 110 is computationally universal (can simulate any Turing machine), the UWCA is also computationally universal. This is machine-certified in Lean 4 (`uwca_sweep_implements_rule110`, zero sorry).

3. **The polynomial is the machine.** The GTE polynomial $p(L, C, R) = C + R - CR - LCR$ is not just a formula for a lookup table. It *is* the update mechanism. The arithmetic of the polynomial, over the 7-element field, is what the UWCA computes. The 7-state behaviour and the 2-state (Rule 110) behaviour are the same polynomial, evaluated over different input ranges.

4. **Minimum description.** Specifying the UWCA requires specifying the polynomial, which requires 19 bits. A direct Rule 110 lookup table requires 8 bits. The UWCA is slightly more expensive to specify than bare Rule 110 — but it operates over 7 states and encodes far more physics. Among all 7-state rules consistent with the physical constraints, the polynomial is the *minimum* description.

9 Summary

The UWCA in plain English

1. Every cell position gets its own prime number. Cell states are encoded as residues (remainders) of a big number, one per prime.

2. At each step, a window slides across the tape. For each position, the UWCA reads the three neighbours' states as residues, evaluates the polynomial $p(L, C, R)$ mod 7, and finds the unique new state that makes the penalty zero.

3. Local determinism (a theorem) guarantees each position has exactly one valid new state — so the scan never gets stuck.

4. In the binary sub-sector (all states 0 or 1), the polynomial output matches Rule 110's truth table exactly. The UWCA therefore implements Rule 110 — and since Rule 110 is computationally universal, the UWCA is too.

5. The polynomial is not a design choice. It is selected by minimum description length from all 7-state, radius-1 rules, and the Standard Model's own algebraic structure forces it to restrict to Rule 110 in the binary case.

For the full formal treatment: the UWCA construction and GTE-as-program compilation theorem are in the P08 monograph; the triangle closure (Direct-Interpolation Lift, chirality census, and the GTE-as-UWCA section) is in the P48 [2] synthesis monograph. Both are available at

novaspivack.com/research. The machine-certified Lean proofs are in the `ugp-lean` repository (doi.org/10.5281/zenodo.20560550).

Further Reading

Primary Sources

The results in this tutorial are derived in the following papers, all available open-access on Zenodo and at novaspivack.com/research:

- **P28 — Computational Universality** (UWCA construction): doi.org/10.5281/zenodo.20259513
- **P48 — The Complete GTE Framework** (main synthesis monograph): doi.org/10.5281/zenodo.20560550

Lean 4 machine proofs: github.com/novaspivack/ugp-lean

References

[1] Nova Spivack. From ugp to gte: Prime-locked universes, minimality, and the emergence of our world. Zenodo, 2026. URL https://doi.org/10.5281/zenodo.20171002. https://doi.org/10.5281/zenodo.20171002. UGP Physics Programme, Paper 08 (P08).

[2] Nova Spivack. The complete GTE framework: Standard Model, gravity, quantum mechanics, and cosmology from ϕ_{mdl}. Zenodo, 2026. URL https://doi.org/10.5281/zenodo.20560550. https://doi.org/10.5281/zenodo.20560550. Preprint.